AI 智能体技术研究

研究主线

```
Prompt Engineering ──→ Context Engineering ──→ Harness Engineering
   "写好一句话"              "管理整个上下文"              "构建智能体运行框架"
    (2022–2023)              (2024–2025)                (2025–2026)
```

三个阶段的核心区别：

维度	Prompt Engineering	Context Engineering	Harness Engineering
关注点	单次 prompt 的质量	整个 token 集合的编排	智能体的完整运行环境
优化对象	指令文本	上下文窗口中的所有信息	工具、流程、协作、持久化
类比	写一封好邮件	管理整个会议议程	设计整个公司运营体系
复杂度	低	中	高

第一部分：PROMPT ENGINEERING（提示词工程）

1.1 基础理论

- LLM 的 in-context learning 机制
- Token 与 attention 的基本原理
- 为什么 prompt 的措辞会影响输出质量

> 关键文献
>
> - R01 Attention Is All You Need — Vaswani et al., Google Brain, 2017 总分 13.4
>   Transformer 架构的奠基论文。提出 Self-Attention 机制，取代 RNN/CNN 成为 NLP 主流架构，是所有现代 LLM 的基石。
> - R02 Language Models are Few-Shot Learners — Brown et al., OpenAI, 2020 (GPT-3) 总分 14.2
>   证明了大规模语言模型具备 in-context learning 能力——仅通过 prompt 中的少量示例就能完成新任务，无需微调。催生了 Prompt Engineering 这个领域。

1.2 核心技术

- Zero-shot prompting：无示例直接提问
- Few-shot prompting：提供示例引导
- Chain-of-Thought (CoT)：引导逐步推理（Wei et al., 2022）
- Self-Consistency：多次采样取多数（Wang et al., 2023）
- Tree of Thoughts (ToT)：树状搜索推理路径（Yao et al., 2023）
- ReAct：推理与行动交替（Yao et al., 2023）

> 关键文献
>
> - R04 Chain-of-Thought Prompting Elicits Reasoning in LLMs — Wei et al., Google Brain, 2022 总分 14.0
>   提出 CoT prompting：在 prompt 中加入逐步推理示例，让 LLM 生成中间推理步骤，算术推理准确率从 17.9% 提升到 58.1%。
> - R05 Self-Consistency Improves Chain of Thought Reasoning — Wang et al., Google, 2023 总分 12.0
>   对 CoT 的改进：多次采样不同推理路径，通过多数投票选择最一致的答案。在 GSM8K 上将 CoT 的 56.5% 提升到 74.4%。
> - R06 Tree of Thoughts: Deliberate Problem Solving with LLMs —

Yao et al., Princeton, 2023 总分 11.2
将 LLM 推理从线性链扩展为树状搜索。模型可以探索多条推理路径、回溯、评估，适合需要前瞻和规划的复杂问题。

- R07 ReAct: Synergizing Reasoning and Acting in LLMs — Yao et al., Princeton + Google, 2023 总分 14.0
提出推理与行动交替的范式。LLM 先思考下一步该做什么，再执行动作（如搜索），用结果继续推理。是 Agent 工具调用的理论基础。

1.3 实用框架

- 角色设定（Role prompting）
- 结构化输出（JSON mode、XML tags）
- 提示词模板与复用
- 提示词版本管理

关键文献

- R10 OpenAI Prompt Engineering Guide — OpenAI, 持续更新 总分 13.0
官方提示词工程指南。涵盖六大策略：写清晰指令、提供参考文本、拆分复杂任务、给模型思考时间、使用外部工具、系统化测试。
- R11 Anthropic Prompt Engineering Guide — Anthropic, 持续更新 总分 12.0
强调 XML 标签结构化、预填充回复、思维链、长上下文技巧。包含 Claude 特有的最佳实践。
- R09 Principled Instructions Are All You Need — Bsharat et al., VILA Lab, 2024 总分 10.0
系统测试了 26 条提示词原则在 LLaMA/GPT-4 上的效果。对"No need to be polite""Think step by step"等经验法则进行了量化验证。

1.4 局限性

- 单次调用的天花板——复杂任务无法靠一条 prompt 搞定
- Prompt 脆弱性——微小改动导致输出巨变
- 缺乏状态管理——每次调用都是无状态的
- → 催生了 Context Engineering 的需求

关键文献

- R08 Calibrate Before Use: Improving Few-Shot Performance — Zhao et al., Stanford, 2021 总分 9.0
  揭示 Few-shot prompting 的三大偏差：多数标签偏差、近因偏差、常见词偏差。微小的示例顺序变化可导致准确率从 50% 到 93% 的巨大波动。

第二部分：CONTEXT ENGINEERING（上下文工程）

参考：Anthropic「Effective Context Engineering for AI Agents」(2025.09)

2.1 从 Prompt 到 Context 的范式转变

- Prompt Engineering 关注"如何写指令"
- Context Engineering 关注"推理时的整个 token 集合"
- 核心问题："什么样的上下文配置最有可能引导模型产生期望行为？"

关键文献

- R12 Effective Context Engineering for AI Agents — Anthropic Applied AI 团队, 2025.09 总分 15.0
  Context Engineering 的定义性文章。提出"上下文工程是提示词工程的自然演进"，系统阐述了上下文写入、压缩、隔离策略。将上下文定义为"推理时的全部 token 集合"。
- R13 The Rise of Context Engineering — Toby Mao (Cursor CTO), 2025.06 总分 11.0
  提出 Context Engineering 不只是 prompting，而是"为 LLM 的每次推理构建动态的、最优的信息环境"。引发了业界对这一概念的广泛讨论。

2.2 为什么需要上下文工程

- **注意力预算**：LLM 的注意力是有限资源，n 个 token 产生 n^2 的注意力关系
- **上下文腐蚀**：token 数量增加 → 回忆准确率下降
- **位置编码的局限**：长序列中位置理解能力下降

> 关键文献
>
> - R14 Lost in the Middle: How LLMs Use Long Contexts — Liu et al., Stanford, 2023 总分 13.2
> 发现 LLM 在长上下文中存在"U 形注意力"——对开头和末尾的信息回忆好，中间位置的信息容易被遗忘。直接影响了上下文编排策略。

2.3 上下文的四大组成

1. **系统提示词（System Prompt）**
    - 清晰度 vs 灵活度的平衡
    - XML/Markdown 分区组织
    - 迭代式提示词开发
2. **工具定义（Tools）**
    - 工具即上下文——每个工具描述占用 token
    - 最小可行工具集原则
    - 工具描述的精确性
3. **检索增强（RAG / 动态上下文）**
    - Agent-as-retriever 模式
    - 搜索质量 vs 上下文膨胀的权衡
4. **对话历史（Conversation History）**
    - 上下文压缩策略
    - 摘要 vs 滑动窗口 vs 选择性保留

> 关键文献
>
> - R03 Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks — Lewis et al., Meta AI, 2020 总分 14.0
> 提出 RAG 范式：将外部知识检索与生成模型结合，模型先检索相关文

档片段，再基于检索结果生成回答。减少幻觉，增强事实性。

- R15 📘 Building Effective Agents — Anthropic, 2024.12 总分 14.0
  Anthropic 的 Agent 构建方法论。提出"从简单开始"原则：优先用增强型 LLM 调用，只在真正需要时才升级到完整 Agent。定义了 Workflow vs Agent 的关键区别。
- R16 📘 Prompt Caching with Claude — Anthropic, 2024.08 总分 10.6
  介绍 Prompt Caching 技术：缓存重复使用的长上下文前缀，减少 90% 的延迟和成本。对上下文工程的实际部署有重大影响。

2.4 上下文管理策略

- 上下文写入：决定哪些信息进入上下文
  - 系统提示词设计
  - 工具结果过滤
  - 文件内容的选择性加载
- 上下文压缩：在信息量和 token 数之间权衡
  - 对话摘要
  - 工具输出截断
  - 冗余信息去除
- 上下文隔离：防止不同任务间的信息污染
  - 子 Agent 独立上下文
  - Worktree 隔离

关键文献

- R17 📄 LLMLingua: Compressing Prompts for Accelerated Inference — Jiang et al., Microsoft, 2023 总分 11.0
  提出用小模型对 prompt 进行压缩，保留关键语义信息的同时减少 token 数量。可实现 20x 压缩且保持 90%+ 的原始性能。
- R18 📗 Long Context Prompting Tips — Anthropic, 持续更新 总分 9.6
  关于长上下文使用的最佳实践。包括文档位置策略（重要内容放前面）、XML 标签分隔、多文档场景处理技巧。

2.5 实践案例

- Claude Code 的上下文管理机制（CLAUDE.md、memory、task system）
- Cursor / Windsurf 的上下文策略对比
- MCP (Model Context Protocol) 作为上下文连接标准

> 关键文献
>
> - R19 Model Context Protocol Specification — Anthropic, 2024.11 开源 总分 14.0
>   Agent 与外部工具/数据源的标准化连接协议。定义了 Resources、Tools、Prompts 三类原语，正在成为 Agent 工具连接的事实标准。已被 Cursor、Windsurf、Zed 等采纳。
> - R20 Claude Code: Best Practices for Agentic Coding — Anthropic, 2025 总分 12.8
>   Claude Code 的官方最佳实践。详述 CLAUDE.md 配置、Memory 系统、子 Agent 上下文隔离、Task 管理等上下文工程的具体实现。

第三部分：HARNESS ENGINEERING（框架工程）

> 参考：gstack（Garry Tan）、Claude Code 架构、Agentic Design Patterns

3.1 什么是 Harness Engineering

- **定义**：为 AI 智能体构建完整运行框架的工程实践
- 超越上下文管理——包括工具编排、流程控制、质量保障、持久化、协作
- 核心理念：AI 的能力上限不取决于模型本身，而取决于它运行的"框架"
- 如果说 Context Engineering 是"管理会议议程"，Harness Engineering 就是"设计整个公司"

3.2 Harness 的核心组件

3.2.1 工具系统（Tool System）

- 工具的定义、注册与发现
- MCP Server 集成
- 工具权限与安全沙箱
- 工具调用的错误处理与重试

> 关键文献
>
> - R21 Toolformer: LMs Can Teach Themselves to Use Tools — Schick et al., Meta AI, 2023 总分 13.2
>   证明 LLM 可以自主学习何时以及如何调用外部工具（计算器、搜索引擎、翻译器等）。是 Agent 工具调用能力的理论基础。
> - R22 Gorilla: Large Language Model Connected with Massive APIs — Patil et al., UC Berkeley, 2023 总分 12.0
>   训练 LLM 精确调用 1,600+ 个 API。发现 Retrieval-Aware Training 可大幅减少 API 调用的幻觉问题。
> - R23 Anthropic Claude Agent SDK — Anthropic, 2025 开源 总分 12.8
>   官方 Agent 开发框架。提供 Tool、Handoff、Guardrail 三大原语，支持 Multi-Agent 编排和安全护栏。

3.2.2 流程编排（Workflow Orchestration）

- 单 Agent 的任务分解与执行
- Multi-Agent 协作模式（参考 Agentic Design Patterns 21 种模式）
  - Prompt Chaining（串联）
  - Routing（路由）
  - Parallelization（并行）
  - Orchestrator-Worker（编排者-工人）
- 人机协作节点（Human-in-the-Loop）

> 关键文献

> - R24 Agentic Design Patterns (Springer 2025) — Antonio Gulli, 2025 总分 13.8
>   系统化的 21 种 Agent 设计模式。从 Prompt Chaining 到 Multi-Agent，按复杂度递增编排。配套 LangChain/CrewAI/Google ADK 代码。
> - R25 Andrew Ng: Agentic Design Patterns — Andrew Ng, DeepLearning.AI, 2024 总分 12.2
>   提出四种核心 Agentic Pattern：Reflection、Tool Use、Planning、Multi-Agent。影响了整个 Agent 开发社区对设计模式的认知。
> - R26 Reflexion: Language Agents with Verbal Reinforcement Learning — Shinn et al., Northeastern, 2023 总分 12.0
>   Agent 从失败中学习的机制：将错误经验转化为自然语言反思，存入记忆，指导后续行为。HumanEval 上从 80% 提升到 91%。

3.2.3 记忆与持久化（Memory & Persistence）

- 短期记忆：对话上下文
- 中期记忆：任务状态、TODO list
- 长期记忆：用户偏好、项目知识、反馈记录
- 持久化存储：文件系统 vs 向量数据库 vs 结构化 DB

3.2.4 质量保障（Quality Assurance）

- 输出验证与自检（Reflection 模式）
- 自动化测试（gstack 的 `/qa` 三级测试体系）
- 代码审查（`/review` Staff Engineer 模式）
- Fix-first 哲学：发现问题直接修，不只是报告

3.2.5 安全与权限（Security & Permissions）

- 工具调用的权限分级
- 危险操作确认机制
- 沙箱执行环境
- 密钥与敏感信息管理

3.2.6 可观测性（Observability）

- Agent 行为日志
- Token 使用追踪
- 任务执行耗时分析
- 错误与降级监控

3.3 典型 Harness 架构案例

案例一：Claude Code

- 架构：单 Agent + 子 Agent 模式
- 上下文管理：CLAUDE.md（项目指令）+ Memory（长期记忆）+ Task（任务追踪）
- 工具系统：Read/Write/Edit/Bash/Glob/Grep + MCP
- 权限模型：自动允许 / 需确认 / 拒绝
- Hooks 系统：事件驱动的自动化

案例二：gstack（Garry Tan）

- 架构：21 个 slash 命令 = 21 个专家角色
- 流程：Think → Plan → Build → Review → Test → Ship → Reflect
- 核心原则："Boil the Lake" 完整性、Fix-first、原子 commit
- 人格化设计：每个角色有明确的行为特征和红线

案例三：Cursor / Windsurf

- 架构：IDE 内嵌 Agent
- 上下文策略：代码库索引 + 文件引用 + 符号分析
- 差异化：与编辑器深度集成（diff view、inline edit）

案例四：Devin / OpenHands

- 架构：自主式全栈 Agent
- 环境：完整的 VM/容器运行环境
- 挑战：自主性 vs 可控性的平衡

关键文献

- R27 SWE-agent: Agent-Computer Interfaces for Software Engineering — Yang et al., Princeton, 2024 总分 14.0
  提出 Agent-Computer Interface (ACI) 概念——Agent 与代码环境的交互界面设计。在 SWE-bench 上实现 12.5% 的 resolve rate，推动了编程 Agent 的发展。
- R28 gstack: Virtual Engineering Team — Garry Tan (YC CEO), GitHub, 2025 总分 11.8
  21 个 Claude Code slash 命令组成的虚拟工程团队。Think→Plan→Build→Review→Test→Ship→Reflect 完整 sprint 流程。"Boil the Lake" 完整性原则。
- R29 OpenAI Swarm (Experimental) — OpenAI, GitHub, 2024 总分 11.0
  OpenAI 的实验性 Multi-Agent 编排框架。核心概念：Agent 间通过 Handoff 转移控制权。轻量级设计，强调教学目的而非生产使用。

3.4 Harness 设计原则

1. **最小权限原则**：Agent 只拥有完成任务所需的最小权限
2. **渐进信任**：从需确认到自动允许的信任升级路径
3. **可逆优先**：优先选择可撤销的操作
4. **人类保持控制权**：关键决策点保留人类介入能力
5. **透明性**：Agent 的每一步操作对用户可见
6. **韧性**：优雅处理失败，而非静默崩溃

关键文献

- R30 Practices for Governing Agentic AI Systems — OpenAI, 2023.12 总分 11.2
  OpenAI 提出的 Agent 治理框架。核心原则：Agent 的权限应限于完成任务所需最小范围，关键决策点须保留人类控制权。
- R31 Anthropic Responsible Scaling Policy — Anthropic, 2024 总分 9.4
  定义 AI Safety Level (ASL) 分级，明确不同能力等级的 Agent 需满足的安全要求。

第四部分：前沿趋势与交叉研究

4.1 Agent-to-Agent 协议

- Anthropic MCP vs Google A2A
- Agent 间通信标准的竞争
- 未来的 Agent 互操作生态

> 关键文献
>
> - R32 Google Agent-to-Agent (A2A) Protocol — Google, 2025.04 总分 11.0
>   Agent 间通信协议。与 MCP（Agent-to-Tool）互补，聚焦于 Agent-to-Agent 场景：任务委派、状态同步、能力发现。
> - R33 OpenAI Agents SDK — OpenAI, 2025 总分 11.8
>   OpenAI 的正式 Agent SDK（Swarm 的生产版）。支持 Tool、Handoff、Guardrail，内置 tracing。与 Anthropic Agent SDK 形成竞争。

4.2 Agent 的自我进化

- 从 Reflection 到 Learning & Adaptation
- Agent 能否自己优化自己的 Prompt / Context / Harness?
- Meta-Agent：管理其他 Agent 的 Agent

> 关键文献
>
> - R34 Voyager: An Open-Ended Embodied Agent with LLMs — Wang et al., NVIDIA, 2023 总分 13.4
>   Minecraft 中的自主学习 Agent。通过自动课程设计、技能库积累和代码生成实现持续自我进化，不需要人类干预即可解锁新能力。
> - R35 Self-Refine: Iterative Refinement with Self-Feedback — Madaan et al., CMU, 2023 总分 12.0
>   LLM 自我反馈-自我改进的迭代框架。模型生成输出→自我评价→自我改

> 进，无需外部监督。代码生成、推理等任务平均提升 20%。

4.3 领域特化的 Harness

- 编程 Agent（Claude Code、Cursor、Devin）
- 研究 Agent（Deep Research、STORM）
- 数据分析 Agent
- 创作 Agent（写作、设计、视频）

> 关键文献
>
> - R36 SWE-bench: Can LMs Resolve Real-World GitHub Issues? — Jimenez et al., Princeton, 2024 总分 13.2
>   编程 Agent 的标准评估基准。包含 2,294 个真实 GitHub issue，要求 Agent 定位代码、生成补丁、通过测试。推动了整个编程 Agent 赛道。
> - R37 WebArena: A Realistic Web Environment for Autonomous Agents — Zhou et al., CMU, 2023 总分 11.2
>   真实网页环境中的 Agent 基准测试。Agent 需在购物、论坛、代码管理等网站上完成 812 个复杂任务。GPT-4 当时仅完成 14.4%。
> - R38 STORM: Synthesis of Topic Outlines through Retrieval and Multi-perspective QA — Shao et al., Stanford, 2024 总分 12.0
>   自动化长文研究写作 Agent。先通过多角色模拟「头脑风暴」生成问题，再检索和整合信息，生成类似维基百科质量的长文。

4.4 评估与基准测试

- SWE-bench、WebArena 等 Agent 基准
- 如何评估 Harness 的质量?
- Agent 可靠性的量化指标

> 关键文献
>
> - R39 SWE-bench Verified — OpenAI + Princeton, 2024 总分 11.0
>   人工验证的 500 个高质量问题版本，消除了原始数据集中的噪声，作为更可靠的 Agent 能力指标。

4.5 经济学视角

- Agent 的成本结构：token 费用 + 工具调用 + 人类监督
- 何时使用 Agent vs 何时使用传统工具
- Agent 的 ROI 计算

> 关键文献
>
> - R40 📘 The Shift from Models to Compound AI Systems — Zaharia et al., Berkeley AI Research, 2024 总分 13.2
>   提出"复合 AI 系统"概念：最前沿的 AI 不再是单个模型，而是模型+检索+工具+代码的系统工程。预示了 Harness Engineering 的必然性。

第五部分：研究计划

阶段一：文献综述（深度阅读）

奠基性论文

- [ ] Vaswani et al.「Attention Is All You Need」(2017)
- [ ] Brown et al.「Language Models are Few-Shot Learners」(GPT-3, 2020)
- [ ] Lewis et al.「Retrieval-Augmented Generation」(2020)

Prompt Engineering 核心论文

- [ ] Wei et al.「Chain-of-Thought Prompting」(2022)
- [ ] Wang et al.「Self-Consistency」(2023)
- [ ] Yao et al.「Tree of Thoughts」(2023)
- [ ] Yao et al.「ReAct」(2023)
- [ ] Zhao et al.「Calibrate Before Use」(2021)
- [ ] Bsharat et al.「Principled Instructions Are All You Need」(2024)

Context Engineering

- Anthropic「Effective Context Engineering for AI Agents」(2025)
- Anthropic「Building Effective Agents」(2024)
- Liu et al.「Lost in the Middle」(2023)
- Jiang et al.「LLMLingua: Compressing Prompts」(2023)
- Toby Mao「The Rise of Context Engineering」(2025)

Harness Engineering

- 《Agentic Design Patterns》全书精读 (Gulli, Springer 2025)
- Andrew Ng「Agentic Design Patterns」系列 (2024)
- Schick et al.「Toolformer」(2023)
- Patil et al.「Gorilla」(2023)
- Shinn et al.「Reflexion」(2023)
- Yang et al.「SWE-agent」(2024)
- Garry Tan gstack 源码分析
- Claude Code 架构与源码分析
- Anthropic Claude Agent SDK 文档
- OpenAI Swarm / Agents SDK 文档

前沿与安全

- Wang et al.「Voyager」(NVIDIA, 2023)
- Madaan et al.「Self-Refine」(CMU, 2023)
- Shao et al.「STORM」(Stanford, 2024)
- Zaharia et al.「The Shift from Models to Compound AI Systems」(Berkeley, 2024)
- OpenAI「Practices for Governing Agentic AI Systems」(2023)
- Anthropic「Responsible Scaling Policy」(2024)
- Jimenez et al.「SWE-bench」(Princeton, 2024)
- Zhou et al.「WebArena」(CMU, 2023)
- Andrej Karpathy 关于 AI 的系列观点

阶段二：横向对比

- [ ] 主流 Agent 框架对比（LangChain/LangGraph、CrewAI、Google ADK、AutoGen）
- [ ] 主流编程 Agent 对比（Claude Code、Cursor、Windsurf、Devin）
- [ ] Prompt vs Context vs Harness 三者关系的理论化

阶段三：实践验证

- [ ] 构建一个最小化的 Harness Engineering demo
- [ ] 在实际项目中测试不同上下文策略的效果
- [ ] 设计一套 Agent 质量评估框架

阶段四：输出成果

- [ ] 系列文章：从 Prompt 到 Context 到 Harness 的技术演进
- [ ] 课程大纲：面向开发者的 AI 智能体工程课程
- [ ] 开源项目：可复用的 Harness Engineering 组件

参考文献总表

> 评分说明（每项 1-5 分）
>
> - **深度**：内容的技术深度与系统性
> - **影响**：引用量、业界采纳度、对后续研究的推动力
> - **实用**：对实际 Agent 开发的直接帮助
> - **总分**：三项加权平均（影响 ×1.2 + 深度 ×1.0 + 实用 ×0.8），满分 15

一、奠基性论文

#	文献	作者 / 机构	年份	类型	深度	影响	实用	总分	链接
R01	Attention Is All You Need	Vaswani et al. / Google Brain	2017	论文	5	5	3	13.4	arXiv:1706.03762
R02	Language Models are Few-Shot Learners (GPT-3)	Brown et al. / OpenAI	2020	论文	5	5	4	14.2	arXiv:2005.14165
R03	Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks	Lewis et al. / Meta AI	2020	论文	4	5	5	14.0	arXiv:2005.11401

二、Prompt Engineering

#	文献	作者 / 机构	年份	类型	深度	影响	实用	总分	链接
R04	Chain-of-Thought Prompting Elicits Reasoning in LLMs	Wei et al. / Google Brain	2022	论文	4	5	5	14.0	arXiv:2201.11903
R05	Self-Consistency Improves Chain of Thought Reasoning	Wang et al. / Google	2023	论文	4	4	4	12.0	arXiv:2203.11171
R06	Tree of Thoughts: Deliberate Problem Solving with LLMs	Yao et al. / Princeton	2023	论文	4	4	3	11.2	arXiv:2305.10601
R07	ReAct: Synergizing Reasoning and Acting in LLMs	Yao et al. / Princeton + Google	2023	论文	4	5	5	14.0	arXiv:2210.03629
R08	Calibrate Before Use: Improving Few-Shot Performance	Zhao et al. / Stanford	2021	论文	3	3	3	9.0	arXiv:2102.09690
R09	Principled Instructions Are All You Need	Bsharat et al. / VILA Lab	2024	论文	3	3	4	10.0	arXiv:2312.16171
R10	OpenAI Prompt Engineering Guide	OpenAI	持续更新	指南	3	5	5	13.0	platform.openai.com
R11	Anthropic Prompt Engineering Guide	Anthropic	持续更新	指南	3	4	5	12.0	docs.anthropic.com

三、Context Engineering

#	文献	作者 / 机构	年份	类型	深度	影响	实用	总分	链接
R12	Effective Context Engineering for AI Agents	Anthropic Applied AI 团队	2025	博客	5	5	5	15.0	anthropic.com
R13	The Rise of Context Engineering	Toby Mao / Cursor CTO	2025	博客	3	4	4	11.0	tobyma.com
R14	Lost in the Middle: How LLMs Use Long Contexts	Liu et al. / Stanford	2023	论文	4	5	4	13.2	arXiv:2307.03172
R15	Building Effective Agents	Anthropic	2024	博客	4	5	5	14.0	anthropic.com
R16	Prompt Caching with Claude	Anthropic	2024	博客	3	3	5	10.6	docs.anthropic.com
R17	LLMLingua: Compressing Prompts for Accelerated Inference	Jiang et al. / Microsoft	2023	论文	4	3	4	11.0	arXiv:2310.05736
R18	Long Context Prompting Tips	Anthropic	持续更新	指南	2	3	5	9.6	docs.anthropic.com
R19	Model Context Protocol Specification	Anthropic	2024	规范	4	5	5	14.0	modelcontextprotocol.io
R20	Claude Code: Best Practices for Agentic Coding	Anthropic	2025	博客	4	4	5	12.8	anthropic.com

四、Harness Engineering

#	文献	作者 / 机构	年份	类型	深度	影响	实用	总分	链接
R21	Toolformer: LMs Can Teach Themselves to Use Tools	Schick et al. / Meta AI	2023	论文	4	5	4	13.2	arXiv:2302.04761
R22	Gorilla: Large Language Model Connected with Massive APIs	Patil et al. / UC Berkeley	2023	论文	4	4	4	12.0	arXiv:2305.15334
R23	Anthropic Claude Agent SDK	Anthropic	2025	开源	4	4	5	12.8	github.com/anthropics
R24	Agentic Design Patterns (Springer 2025)	Antonio Gulli	2025	专著	5	4	5	13.8	Springer
R25	Andrew Ng: Agentic Design Patterns	Andrew Ng / DeepLearning.AI	2024	博客	3	5	4	12.2	deeplearning.ai
R26	Reflexion: Language Agents with Verbal Reinforcement Learning	Shinn et al. / Northeastern	2023	论文	4	4	4	12.0	arXiv:2303.11366
R27	SWE-agent: Agent-Computer Interfaces for Software Engineering	Yang et al. / Princeton	2024	论文	4	5	5	14.0	arXiv:2405.15793
R28	gstack: Virtual Engineering Team	Garry Tan / YC CEO	2025	开源	3	4	5	11.8	github.com/garrytan
R29	OpenAI	OpenAI	2024		3	4	4	11.0	github.com/openai

	Swarm (Experimental)			开源					
R30	Practices for Governing Agentic AI Systems	OpenAI	2023	博客	4	4	3	11.2	openai.com
R31	Anthropic Responsible Scaling Policy	Anthropic	2024	博客	3	4	2	9.4	anthropic.com

五、前沿趋势

#	文献	作者 / 机构	年份	类型	深度	影响	实用	总分	链接
R32	Google Agent-to-Agent (A2A) Protocol	Google	2025	规范	3	4	4	11.0	google.github.io/A2A
R33	OpenAI Agents SDK	OpenAI	2025	开源	3	4	5	11.8	github.com/openai
R34	Voyager: An Open-Ended Embodied Agent with LLMs	Wang et al. / NVIDIA	2023	论文	5	5	3	13.4	arXiv:2305.16291
R35	Self-Refine: Iterative Refinement with Self-Feedback	Madaan et al. / CMU	2023	论文	4	4	4	12.0	arXiv:2303.17651
R36	SWE-bench: Can LMs Resolve Real-World GitHub Issues?	Jimenez et al. / Princeton	2024	论文	4	5	4	13.2	arXiv:2310.06770
R37	WebArena: A Realistic Web Environment	Zhou et al. / CMU	2023	论文	4	4	3	11.2	arXiv:2307.13854

	for Autonomous Agents								
R38	STORM: Synthesis of Topic Outlines through Retrieval and Multi-perspective QA	Shao et al. / Stanford	2024	论文	4	4	4	12.0	arXiv:2402.14207
R39	SWE-bench Verified	OpenAI + Princeton	2024	博客	3	4	4	11.0	openai.com
R40	The Shift from Models to Compound AI Systems	Zaharia et al. / Berkeley AI	2024	博客	4	5	4	13.2	bair.berkeley.edu

---

影响力排行 TOP 10

排名	#	文献	总分	必读理由
1	R12	Effective Context Engineering for AI Agents	15.0	本研究的核心参考，Context Engineering 的定义性文章
2	R02	Language Models are Few-Shot Learners (GPT-3)	14.2	催生了整个 Prompt Engineering 领域
3	R04	Chain-of-Thought Prompting	14.0	最广泛使用的推理增强技术
4	R07	ReAct	14.0	Agent 工具调用的理论基础，几乎所有 Agent 框架都基于此
5	R15	Building Effective Agents	14.0	Anthropic 的 Agent 方法论，Workflow vs Agent 的关键区分
6	R19	Model Context Protocol (MCP)	14.0	Agent 工具连接的事实标准
7	R27	SWE-agent	14.0	Agent-Computer Interface 概念，推动编程 Agent 赛道
8	R03	Retrieval-Augmented Generation (RAG)	14.0	所有 RAG 系统的源头
9	R24	Agentic Design Patterns (Springer)	13.8	最系统的 Agent 设计模式参考
10	R01	Attention Is All You Need	13.4	一切的起点，Transformer 架构奠基